BITCOIN: A PROOF-OF-WORK EXTENSIBLE CONSENSUS STATE

ODA BLORANGE[1]

Abstract. Bitcoin may be formalized as a distributed consensus state maintained by a peer-to-peer network that validates an append-only sequence of proof-of-work blocks. Its security derives from objective block validity, probabilistic leader election by hashpower, and deterministic state transition under shared consensus rules. This paper presents a mathematical description of Bitcoin as a state machine over unspent transaction outputs, studies the role of blockspace as a scarce settlement resource, and characterizes forks, policy surfaces, and client compatibility as problems of distributed coordination.

We then propose a general framework for *extensible node utility*: a Bitcoin-adjacent design space in which nodes may follow supersets of Bitcoin-validating behavior without altering the base validity of Bitcoin blocks. In this framework, higher-layer coordination systems can remain anchored to Bitcoin while introducing additional typed messages, deterministic replay, compressed commitments, and distinct resource-allocation markets upstream of L1 settlement. We call the resulting proposed overlay family *Kapnet*. Its fundamental primitives are submissions, relay admission, weakwork-weighted braid formation, checkpoint knots, deterministic execution, and optional Bitcoin anchoring. Finally, we classify the major client types required to operate such a system and describe their trust, bandwidth, storage, and execution assumptions.

Keywords. Bitcoin, proof of work, distributed systems, consensus, UTXO, blockspace, overlay protocols, state machines
2025 Bitcoin Creds. Primary 68M14; Secondary 68M12, 91A46, 94A17

Contents

1. Introduction 2
2. Distributed Timestamp Server 2
3. Bitcoin State as a UTXO Transition System 3
3.1. Transaction graph and UTXO set 3
3.2. Blocks and chain-induced state 4
4. Unforgeable Costliness and Digital Scarcity 5
5. Public State and Secret Ownership 5
5.1. SegWit and weighted blockspace 6
6. Blockspace Dynamics 6
6.1. Mempool as pre-consensus staging area 6
6.2. Block assembly as constrained optimization 7
7. Forks, Compatibility, and Chain Families 7

8. Extensible Node Utility 8
9. A Formal Extension Framework 8
10. Kapnet as Proposed Overlay Geometry 9
10.1. Primitive sets 9
10.2. Layered finality 10
10.3. Deterministic execution 10
10.4. Blockspace interpretation 10
11. Rooms, Guilds, and Namespaced Coordination 10
12. Client Types 11
12.1. Client capability matrix 11
12.2. Trust surfaces 11
13. Interpretive Layers and Meta-Protocols 12
14. Economic Meaning of Extension 12
15. Conclusion 13
References 13

1. Introduction

Bitcoin is best understood neither as a mere payment application nor as a static database, but as a distributed consensus state: a machine by which mutually distrusting participants converge on one presently accepted history of valid state transitions. The whitepaper frames Bitcoin as a peer-to-peer distributed timestamp server that orders transactions by computational proof and thereby solves double spending without a trusted third party. [1] This description remains the cleanest statement of the system's core achievement.

The present paper pursues three goals. First, it gives a compact mathematical language for Bitcoin state. Second, it separates consensus, policy, and interpretation layers so that extension surfaces may be discussed without conflating them. Third, it proposes a rigorous vocabulary for Bitcoin-adjacent overlays that preserve Bitcoin's base settlement logic while expanding node utility, coordination bandwidth, and expressive throughput. The motivating claim is not that Bitcoin should cease to be conservative, but that its formal structure already admits a broad ecology of external systems that can inherit its finality while competing in distinct pre-settlement resource markets.

2. Distributed Timestamp Server

Definition 2.1 (Network and consensus rules)**.** Let N be a peer-to-peer network of validating nodes. Let

$$C_N = C_{\text{parse}} \cup C_{\text{tx}} \cup C_{\text{block}} \cup C_{\text{chain}}$$

be the set of consensus rules that determine whether messages, transactions, blocks, and competing histories are valid for N.

Definition 2.2 (Proof-of-work rule family)**.** Let $W_R \subset C_N$ denote the subset of rules that enforce proof-of-work validity and heaviest-valid-chain selection. These

rules include target verification, ancestry constraints, and the chain-selection criterion that chooses the valid tip with greatest accumulated work.

Definition 2.3 (Consensus state)**.** A *consensus state* is a tuple

$$\Sigma = (H, \mathcal{U}, \mathcal{M}, \mathcal{P})$$

where H is the accepted block history up to the current tip, $\mathcal{U}$ is the UTXO set induced by H, $\mathcal{M}$ is the node's local mempool, and $\mathcal{P}$ is the node's local policy and implementation environment.

Bitcoin is therefore not merely a ledger of balances but a replicated transition system. New blocks propose state updates; honest validating nodes independently verify those updates under C_N and update Σ if and only if the proposal is valid.

Axiom 2.4 (Deterministic validity)**.** For any fixed C_N, any valid block B and predecessor state Σ, the state update

$$\Sigma \xrightarrow{B} \Sigma'$$

is deterministic.

Remark 2.5. Determinism here refers to *consensus semantics*, not to implementation internals. Different codebases may differ in language, structure, database layout, or optimization while remaining compatible if they compute the same validity relation over the same protocol objects.

Conjecture 2.6 (Implementation plurality)**.** Let $S(N)$ be one implementation of the protocol rules C_N. An alternate implementation $S^*(N)$ remains consensus-compatible with $S(N)$ if it computes the same validity predicate on all relevant transaction, block, and chain inputs.

This is the mathematical core of Bitcoin's free-and-open-source decentralization. The protocol is not reducible to one repository, but neither is it independent of implementation correctness. Multiple clients increase sovereignty and resilience, yet also widen the surface for accidental consensus divergence.

3. Bitcoin State as a UTXO Transition System

3.1. Transaction graph and UTXO set.

Definition 3.1 (Transaction)**.** A Bitcoin transaction is a finite byte string

$$\tau = (\mathrm{ver}, I, O, \mathrm{locktime}, \mathrm{witness})$$

whose inputs I reference prior outputs and whose outputs O create new spendable objects under specified locking conditions.

Definition 3.2 (Outpoint and UTXO)**.** An *outpoint* is a pair

$$\sigma = (\mathrm{txid}, j)$$

identifying output index j in transaction txid. A UTXO is an outpoint not yet consumed by any valid input in the accepted history. The global UTXO set is denoted

$$\mathcal{U} \subseteq \mathrm{Outpoints}.$$

Definition 3.3 (Spend condition)**.** Each UTXO $u \in \mathcal{U}$ is associated with a value

$$v(u) \in \mathbb{N}$$

and a locking predicate

$$\pi_u : \mathcal{W} \to \{0, 1\},$$

where $\mathcal{W}$ denotes admissible witness and unlocking data.

A transaction τ is valid with respect to a UTXO set $\mathcal{U}$ if it satisfies the relevant syntax, non-duplication, value-conservation, script, and consensus constraints, and if all referenced outpoints exist in $\mathcal{U}$.

Definition 3.4 (State transition induced by a transaction)**.** If a valid transaction τ consumes the set of spent UTXOs $I(\tau)$ and creates the set of outputs $O(\tau)$, then the induced UTXO transition is

$$\mathcal{U}' = \big(\mathcal{U} \setminus I(\tau)\big) \cup O(\tau).$$

Proposition 3.5 (Local conservation)**.** *Ignoring subsidy and fees for the moment, any valid transaction preserves total value between spent inputs and created outputs:*

$$\sum_{u \in I(\tau)} v(u) = \sum_{o \in O(\tau)} v(o) + f(\tau),$$

where $f(\tau) \geq 0$ is the transaction fee.

Proof. This is a direct consequence of Bitcoin's validity rules: transaction outputs may not exceed the sum of referenced input amounts, and the difference is interpreted as fee. □

3.2. **Blocks and chain-induced state.**

Definition 3.6 (Block)**.** A block is a tuple

$$B = (h, \tau_0, \tau_1, \ldots, \tau_m)$$

where h is a block header and τ_0 is a coinbase transaction.

Definition 3.7 (Header)**.** The block header may be written

$$h = (nVersion, \text{prevhash}, \text{merkleroot}, nTime, nBits, nNonce).$$

Its proof-of-work validity is the condition

$$\text{SHA256d}(h) \leq T(nBits),$$

where $T(nBits)$ is the target encoded by $nBits$.

Definition 3.8 (Block-induced state update)**.** Given predecessor UTXO set $\mathcal{U}$, a valid block B induces

$$\mathcal{U}_B = \Phi_B(\mathcal{U}),$$

where Φ_B applies the valid transaction transitions in block order and adds the coinbase outputs subject to the subsidy-plus-fee bound.

Definition 3.9 (Blockchain state)**.** For a valid chain

$$H = (B_0, B_1, \ldots, B_n),$$

the resulting consensus UTXO set is

$$\mathcal{U}(H) = \Phi_{B_n} \circ \cdots \circ \Phi_{B_1} \circ \Phi_{B_0}(\varnothing).$$

Remark 3.10. This compositional view is the cleanest mathematical account of Bitcoin state. A node need not retain the entire historical block body for consensus on current spendability; it needs a valid block sequence and a correct present UTXO set derived from it.

4. Unforgeable Costliness and Digital Scarcity

Axiom 4.1 (Finite issuance under proof-of-work schedule)**.** Bitcoin's monetary base is defined by a subsidy process embedded in valid block production and asymptotically bounded by the halving schedule.

Let $R(h)$ denote the block subsidy at height h. In the standard schedule,

$$R(h) = 50 \cdot 10^8 \cdot 2^{-\lfloor h/210000 \rfloor}$$

satoshis, rounded according to integer consensus rules, until subsidy terminates. The total issuance is the finite sum

$$\sum_{h=0}^{6929999} R(h) < 21 \cdot 10^{14}.$$

Definition 4.2 (Difficulty adjustment epoch)**.** A difficulty epoch is a window of 2016 consecutive blocks. If t_k and t_{k+2016} are the timestamps used for adjustment, then the next target is computed from the elapsed ratio relative to two weeks, subject to consensus bounds.

The point is not perfect wall-clock regularity but bounded feedback. Difficulty adjustment stabilizes expected block interval despite fluctuating hashrate, thereby preserving the issuance rhythm and scarcity schedule without appointing a leader or external clock.

Remark 4.3. Bitcoin's economic scarcity is therefore inseparable from its computational scarcity. The system does not merely count coins; it prices the right to append history through irreversible energy expenditure.

5. Public State and Secret Ownership

Bitcoin combines public state with secret control. The state transition graph is globally auditable, while ownership is exercised by the ability to satisfy unlocking predicates.

Definition 5.1 (Ownership)**.** A party is said to *control* a UTXO u if it can produce witness data w such that

$$\pi_u(w) = 1.$$

This leads to a fundamental duality:

$$\text{public state discoverability} \quad + \quad \text{private witness knowledge} \quad = \quad \text{bearer-style digital control.} \tag{5.1}$$

Nodes can verify spendability without knowing private keys. Thus Bitcoin offers a common objective state while leaving control fragmented among secret holders. This is why higher-order contract systems may be built around Bitcoin without altering the base validation logic: the public state is globally legible even when the control surface remains private.

5.1. **SegWit and weighted blockspace.**

SegWit did not alter the basic UTXO transition model; it altered the serialization and accounting of data inside a valid block by discounting witness bytes in the block weight formula.

Definition 5.2 (Weight)**.** For a transaction or block object x, define

$$W(x) = 4 \cdot \text{strippedsize}(x) + \text{witnesssize}(x).$$

A block is valid only if

$$W(B) \leq 4{,}000{,}000.$$

This is best understood as a refinement of blockspace pricing. Blockspace is not a homogeneous storage bucket but a consensus-weighted scarce resource. Once this is recognized, the possibility of alternate payload classes, policy tiers, and overlay commitments becomes mathematically natural.

6. Blockspace Dynamics

6.1. **Mempool as pre-consensus staging area.**

Definition 6.1 (Local mempool)**.** For a node n, let

$$\mathcal{M}_n$$

denote the set of transactions known to n that are valid relative to its current chainstate and accepted under its local relay policy.

Remark 6.2. The mempool is not consensus state. Two honest nodes can maintain different mempools while agreeing perfectly on the current chain tip and UTXO set.

A transaction's lifecycle can therefore be stratified into distinct levels:

$$\text{constructed} \to \text{relayed} \to \text{mempool-accepted} \to \text{mined} \to \text{buried}.$$

Consensus begins at block acceptance, not at gossip visibility.

6.2. **Block assembly as constrained optimization.** A miner selecting transactions for a candidate block solves a constrained optimization problem. Let $\mathcal{M}$ be the miner's available mempool and let each transaction τ have fee $f(\tau)$ and weight $W(\tau)$. Ignoring package effects for the moment, the objective is approximately

$$\max_{S \subseteq \mathcal{M}} \sum_{\tau \in S} f(\tau) \quad \text{subject to} \quad \sum_{\tau \in S} W(\tau) \leq 4{,}000{,}000$$

along with ancestry, validity, and package-dependency constraints.

Definition 6.3 (Blockspace)**.** *Blockspace* is the scarce settlement capacity defined by valid block weight and temporally rationed by proof-of-work block production.

This leads immediately to a useful abstraction.

Definition 6.4 (Settlement density)**.** For a set of confirmed transactions S in a block, define settlement density

$$D(S) = \frac{\sum_{\tau \in S} f(\tau)}{\sum_{\tau \in S} W(\tau)}.$$

Bitcoin's fee market is therefore a market for scarce settlement density under probabilistic block arrival. This fact is the proper starting point for any extension architecture.

7. Forks, Compatibility, and Chain Families

Definition 7.1 (Soft fork)**.** A consensus soft fork $F_S(N)$ is an addition of rules R_S to C_N such that the new valid set of blocks and transactions is a strict subset of the old valid set.

Definition 7.2 (Hard fork)**.** A consensus hard fork $F_H(N)$ is a modification of rules that admits some blocks or transactions previously invalid under C_N.

Remark 7.3. Policy changes are neither soft forks nor hard forks unless they alter the consensus-valid set. This distinction is essential. Relay filters, standardness constraints, indexing conventions, and application-level interpretations are not consensus merely because they affect user experience.

Definition 7.4 (Chain family)**.** A *chain family* is a set

$$\mathfrak{F} = \{(N_i, C_i, W_i, \Sigma_i)\}_{i \in I}$$

of proof-of-work networks that share ancestry, software lineage, or design inheritance while maintaining distinct consensus states.

Bitcoin history already demonstrates that conservation and innovation coexist uneasily. Consensus changes are costly because each change permanently alters path dependence. Once a rule is embedded in history, backing it out is not mere software cleanup; it is a coordination event with potential split consequences.

8. Extensible Node Utility

We now separate two kinds of extension:

1. *Consensus extension*: changing the valid state-transition relation of the base chain.
2. *Utility extension*: increasing what a node can do while preserving base consensus compatibility.

This paper is mainly interested in the second.

Definition 8.1 (Extension rule family)**.** Let E be a higher-layer rule family such that

$$C_E \supseteq C_N$$

as local behavior, but where C_E does not alter the predicate of Bitcoin block validity. Then a node implementing C_E is a *Bitcoin-valid utility-extended node*.

Conjecture 8.2 (Extensible node utility)**.** Bitcoin nodes may expand the practical utility of the network by implementing local supersets of behavior that preserve base Bitcoin consensus while introducing additional indexing, coordination, pricing, storage, execution, and anchoring capabilities.

Examples include mempool analytics, package relay policy, alternate indexing layers, sideband message markets, watchtower logic, covenant simulation, federated overlays, and deterministic replay engines anchored by periodic commitments to Bitcoin.

9. A Formal Extension Framework

We now develop a generic model for Bitcoin-adjacent overlay systems.

Definition 9.1 (Overlay message)**.** An overlay message is a typed object

$$m = (\iota, p, \alpha, \kappa)$$

where ι is identity and hash commitment metadata, p is payload, α is an admission bundle, and κ specifies optional anchoring instructions into Bitcoin.

Definition 9.2 (Admission bundle)**.** The admission bundle α may include any combination of signatures, fees, weakwork, membership proofs, capability tokens, rate limits, or other relay rights recognized by the overlay.

Definition 9.3 (Overlay state)**.** An overlay state is a tuple

$$\Xi = (\mathcal{S}, \mathcal{R}, \mathcal{A}, \rho)$$

where $\mathcal{S}$ is the set of accepted submissions, $\mathcal{R}$ is the relay-visible message graph, $\mathcal{A}$ is the currently active execution or governance context, and ρ is a state commitment root.

Definition 9.4 (Anchor map)**.** An anchor map is a function

$$\eta : \Xi \to \mathcal{B}$$

that compresses some overlay state into a Bitcoin-valid anchoring artifact $\mathcal{B}$, such as a transaction commitment, covenant-compatible template, or other admissible settlement-side object.

The main idea is that Bitcoin need not carry every intermediate expressive event. It need only carry sufficiently small commitments to authenticate the resulting overlay history.

10. Kapnet as Proposed Overlay Geometry

We now instantiate the abstract extension framework in a specific proposed family called Kapnet. The goal is to price coordination before settlement, preserve deterministic replay, and compress large expressive histories into small Bitcoin anchor objects.

10.1. **Primitive sets.**

Definition 10.1 (Submission)**.** A *submission* is the atomic coordination object of Kapnet. Formally,

$$s = (h_s, p_s, d_s, e_s)$$

where h_s is a unique content hash, p_s is the payload, d_s is dependency metadata, and e_s is eligibility evidence.

Definition 10.2 (KapActivity)**.** A *KapActivity* is a typed action represented inside the payload of a submission. Examples include post, reply, vote, validation, proposal, execution request, parameter update, or bridge action.

Definition 10.3 (TXXM)**.** A *TXXM* is a higher-order coordination transaction format transporting one or more KapActivities together with admission proofs and optional Bitcoin anchoring instructions.

Definition 10.4 (Weakwork)**.** Weakwork is a non-consensus proof-of-effort object

$$\omega \in \mathcal{WW}$$

used to rank, admit, or price relay of overlay messages without replacing Bitcoin proof-of-work as the final consensus arbiter.

Definition 10.5 (Braid)**.** A *braid* is a partially ordered candidate history

$$\beta = (V_\beta, E_\beta, w_\beta)$$

whose vertices are accepted submissions, whose edges encode dependency or ordering relations, and whose weight function w_β aggregates criteria such as weakwork, fees, burns, stake, or validation score.

Definition 10.6 (Knot)**.** A *knot* is a checkpoint object

$$K = (\beta^\star, \rho^\star, \chi, \eta(\Xi))$$

consisting of a selected braid winner $\beta^\star$, a resulting state root $\rho^\star$, an execution trace commitment χ, and an optional Bitcoin anchor artifact $\eta(\Xi)$.

10.2. **Layered finality.** Kapnet introduces a richer finality ladder than raw Bitcoin mempool visibility:

observed → admitted → weighted → braided → knotted → anchored → deeply anchored.

Remark 10.7. This ladder should not be collapsed into a binary notion of finality. A message can be economically relevant or socially prominent before it is Bitcoin-anchored, just as a Bitcoin transaction can be visible before it is mined.

10.3. **Deterministic execution.**

Definition 10.8 (Bounded replay machine)**.** A Kapnet execution engine is a deterministic machine

$$\mathsf{Exec} : (\rho, s) \mapsto (\rho', \chi, c)$$

which takes prior state root ρ and submission s and returns a new state root ρ', an execution trace commitment χ, and a bounded resource cost c.

Axiom 10.9 (Bounded execution)**.** There exists a resource bound L such that any replayable Kapnet execution halts within L steps or fails cleanly without ambiguity.

Remark 10.10. This bound is essential. Without it, deterministic overlay state cannot be safely shared among heterogeneous client types.

10.4. **Blockspace interpretation.** Kapnet's central claim about Bitcoin is not that Bitcoin blockspace should carry all coordination, but that it should remain the final thermodynamic sink of compressed commitments. Formally, let

$$\mathcal{E}_{\text{expr}}$$

denote expressive throughput and

$$\mathcal{E}_{\text{set}}$$

denote settlement throughput. Kapnet seeks a regime in which

$$\mathcal{E}_{\text{expr}} \gg \mathcal{E}_{\text{set}}$$

while preserving authenticated reduction from the former to the latter by anchoring commitments.

11. Rooms, Guilds, and Namespaced Coordination

We now introduce the organizational objects most natural to Kapnet-like overlays.

Definition 11.1 (Room)**.** A room is a local coordination domain

$$R = (\mathcal{S}_R, \mathcal{P}_R, \rho_R)$$

with its own submission set, policy parameters, and state root.

Definition 11.2 (Guild)**.** A guild is a higher-order governance object

$$G = (\{R_i\}_{i \in I}, \Gamma, \rho_G)$$

which aggregates rooms, shared rules Γ, and a guild-level state root.

Definition 11.3 (Namespace)**.** A namespace is a recursively composable labeling and authority domain over rooms, guilds, and their derived objects.

These objects do not alter Bitcoin's UTXO validity. They define overlay-local state partitions in which richer forms of coordination may be priced, replayed, and checkpointed.

12. Client Types

The extension framework implies a family of distinct clients. Not all clients must perform the same tasks, but any two clients claiming consensus on the same layer must share the same validity and replay semantics for that layer.

Definition 12.1 (Bitcoin base validator)**.** A *Bitcoin base validator* verifies Bitcoin transactions, blocks, and chainwork under C_N and maintains a correct UTXO set $\mathcal{U}$.

Definition 12.2 (Kapnet relay client)**.** A *Kapnet relay client* participates in propagation, admission, and local ranking of overlay submissions but need not execute all overlay state transitions.

Definition 12.3 (Kapnet execution client)**.** A *Kapnet execution client* replays overlay messages under the deterministic execution rules and maintains overlay roots ρ for one or more rooms or guilds.

Definition 12.4 (Anchor client)**.** An *anchor client* constructs or verifies Bitcoin anchoring artifacts that commit to overlay state, traces, or checkpoints.

Definition 12.5 (Archive client)**.** An *archive client* stores extended historical payloads, braid alternatives, and execution traces beyond what is required for present-state validation.

Definition 12.6 (Light client)**.** A *light client* verifies selected commitments, proofs, or state witnesses without independently retaining or replaying the full underlying data.

12.1. Client capability matrix.

12.2. Trust surfaces.

Summary 12.7. Each client type inhabits a different trust surface:

1. Base validators trust only the correctness of consensus computation.
2. Relay clients trust local policy and peering quality for message visibility.
3. Execution clients trust bounded deterministic replay and state-root construction.
4. Anchor clients trust Bitcoin validity plus the correctness of the commitment scheme.
5. Light clients trust proof systems, data availability assumptions, or delegated witnesses.

Table 1. Illustrative client-type decomposition

Client type	BTC validate	Overlay relay	Overlay execute	Anchor verify	Historical archiv
Bitcoin base validator	✓				
Kapnet relay client	optional	✓		optional	
Kapnet execution client	optional	✓	✓	optional	optional
Anchor client	✓	optional	optional	✓	
Archive client	optional	optional	optional	optional	✓
Light client	optional	optional		proof-based	
Integrated full client	✓	✓	✓	✓	optional

13. Interpretive Layers and Meta-Protocols

A recurring source of confusion in Bitcoin discourse is the failure to distinguish between the following strata:

1. *Consensus-valid data*: what all validating nodes must accept.
2. *Policy-admitted data*: what nodes choose to relay or mine.
3. *Indexed data*: what software chooses to parse and expose.
4. *Interpreted state*: what an application infers from valid data.
5. *Overlay state*: what a higher-layer protocol coordinates and periodically anchors.

Ordinal-style numbering systems, token overlays, and arbitrary parsers belong primarily to layers (3) and (4) unless they alter consensus or are coupled to explicit anchoring contracts. Kapnet, by contrast, is proposed as a more explicit layer-(5) system: it does not merely reinterpret individual satoshis, but organizes a separate coordination state with its own relay market, execution rules, and checkpoint logic.

14. Economic Meaning of Extension

The reason to formalize these distinctions is economic. Bitcoin's native fee market prices final settlement bytes, but digital society produces many forms of action that are too numerous, too contingent, or too conversational to justify direct L1 persistence. An extension network becomes meaningful when it can do all of the following:

1. price attention before settlement,
2. compress many expressive events into few settlement commitments,
3. preserve deterministic replay where necessary,
4. maintain a clear boundary between local policy and shared validity,

5. inherit Bitcoin finality without parasitically replacing Bitcoin's monetary role.

Conjecture 14.1. The most resilient Bitcoin-adjacent overlays will be those that maximize authenticated compression into Bitcoin while minimizing irreversible demands on raw blockspace.

15. Conclusion

Bitcoin is a proof-of-work extensible consensus state. Its essence is not exhausted by the phrase "digital cash," though neither should that phrase be discarded. Bitcoin solves ordered ownership transfer under adversarial conditions by replicating a deterministic UTXO transition system across a permissionless network that selects history by accumulated work. Once formalized this way, several consequences follow.

First, blockspace is properly understood as scarce settlement density rather than mere storage. Second, many disputes often called philosophical are actually disputes about layer separation: consensus versus policy, indexing versus validity, interpretation versus settlement. Third, the possibility of utility extension is native to Bitcoin's public-state architecture. Nodes can do much more than validate blocks while still remaining faithful to base consensus.

Kapnet, as proposed in this paper, names one such extensible direction. It treats coordination messages as first-class typed objects, prices relay and prominence before settlement, organizes them into weighted braid candidates, checkpoint-selects knots, deterministically replays bounded state transitions, and anchors compressed commitments into Bitcoin when final settlement is required. Whether any particular implementation succeeds is an engineering and market question. But the mathematical shape of the design space is already visible: Bitcoin need not directly carry every meaningful action in order to remain the root of authenticated digital history.

Acknowledgment. Special thanks to @itme_brain and @mark_hodl for fueling the contrarian spirit.

References

1. Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System.* https://bitcoin.org/bitcoin.pdf
2. Bitcoin Developer Documentation. *Transactions.* https://developer.bitcoin.org/devguide/transactions.html
3. Walker, G. et al. *UTXO — Unspent Transaction Output.* Learn Me A Bitcoin. https://learnmeabitcoin.com/technical/transaction/utxo/
4. Walker, G. et al. *Merkle Root — A Fingerprint for the Transactions in a Block.* Learn Me A Bitcoin. https://learnmeabitcoin.com/technical/block/merkle-root/
5. Walker, G. et al. *SegWit — What was the Segregated Witness Upgrade?* Learn Me A Bitcoin. https://learnmeabitcoin.com/beginners/guide/segwit/
6. ByteCoin (2011). *Bitcoin Transaction Volume.* Bitcoin Forum, Topic 6172, MSG 90789. https://bitcointalk.org/index.php?topic=6172.msg90789#msg90789
7. Rodarmor, C. (2022). *Ordinal Theory.* https://rodarmor.com/blog/ordinal-theory/

8. Rodarmor, C. (2022). *Inscribing Mainnet*. https://rodarmor.com/blog/inscribing-mainnet/

[1] Founder and Headmaster, Central Bitcoin Timechain Creds
Email address: n0-space@bitcoincreds.com